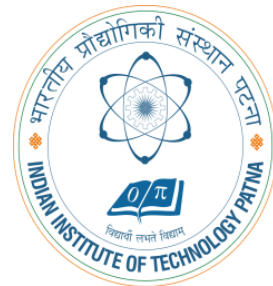


Artificial Intelligence

Lecture 03 - Adversarial Search

Dr. Rajiv Misra, Professor
Dept. of Computer Science & Engg.
Indian Institute of Technology, Patna
rajivm@iitp.ac.in



Competitive and Perfect Information Games

- **Competitive:** Commonly zero sum
 - one player wins, other loses
- **Perfect Information:**
 - Player knows the result of all previous moves
 - There is one best way to play for each player
- **Imperfect Information:**
 - Players do not know all of the previous moves (may play simultaneously)
- **Simple states** for representation – not very complex environment, well known and discrete states

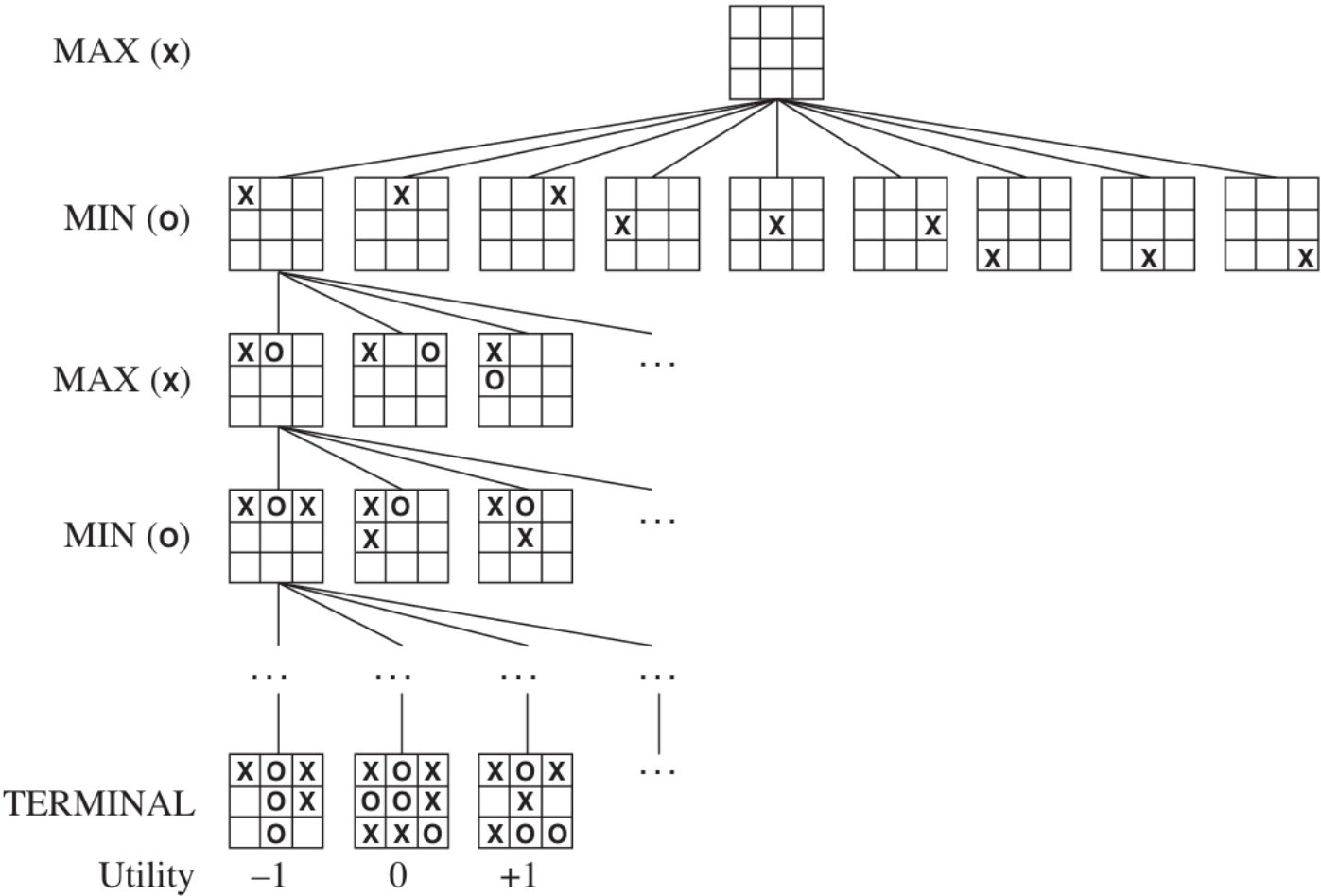
Terminology

- S_0 – initial state
- $\text{player}(s)$ – who's the player in state S
- $\text{action}(s)$ – possible moves from state S
- $\text{result}(s,a)$ – the state after action a is taken on state S
- $\text{terminal}(s)$ – returns true if S is a terminal state
- $\text{utility}(s,p)$ – the objective function in state S for player P

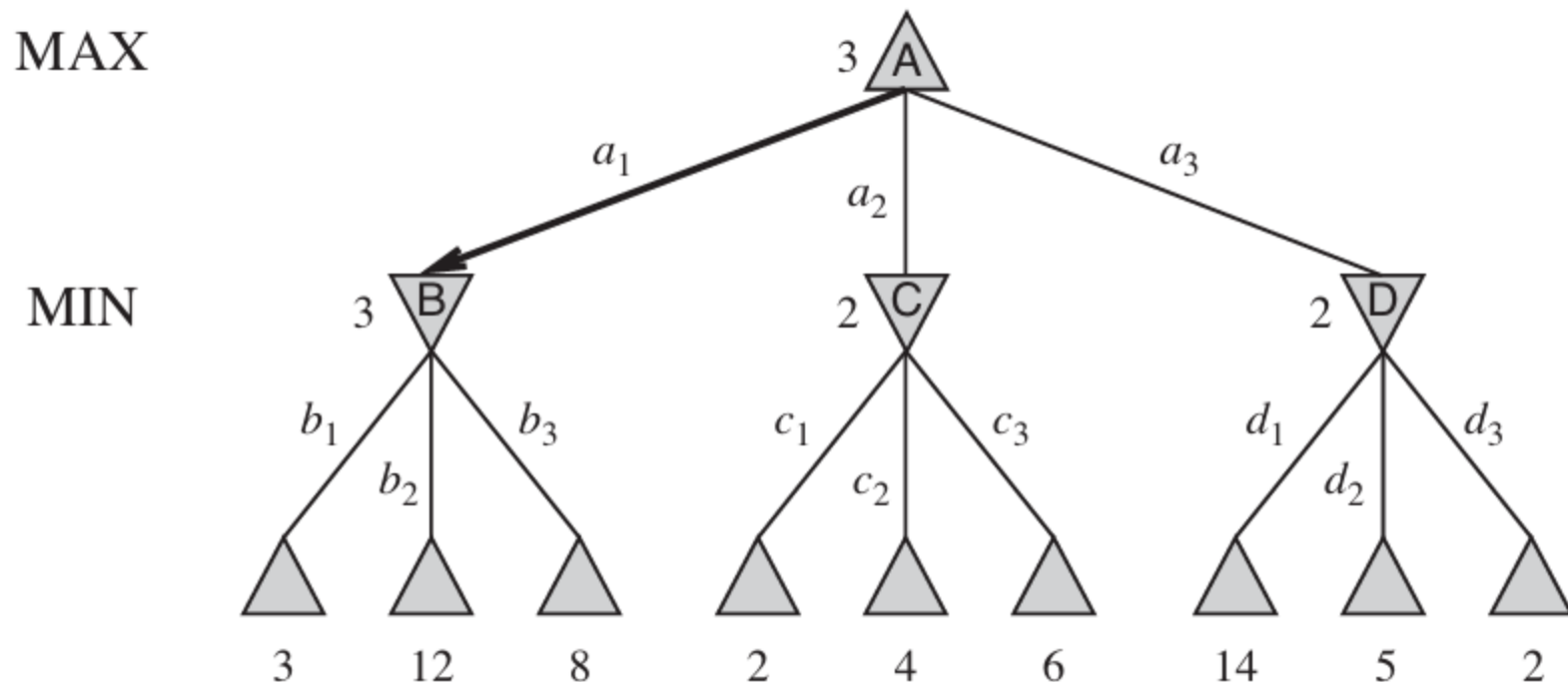
Optimal Decisions – Optimal Players

- A game not only finds best way
 - The other player has a say
-
- Two players: MAX and MIN
 - $\text{actions}(S)$ and $\text{result}(s,a)$ define a game tree
 - Tic Tac Toe: Fewer than $9! = 36288$ terminal nodes
 - Chess: more than 10^{40}
 - Search tree as theoretical foundation

Game Tree – Tic Tac Toe

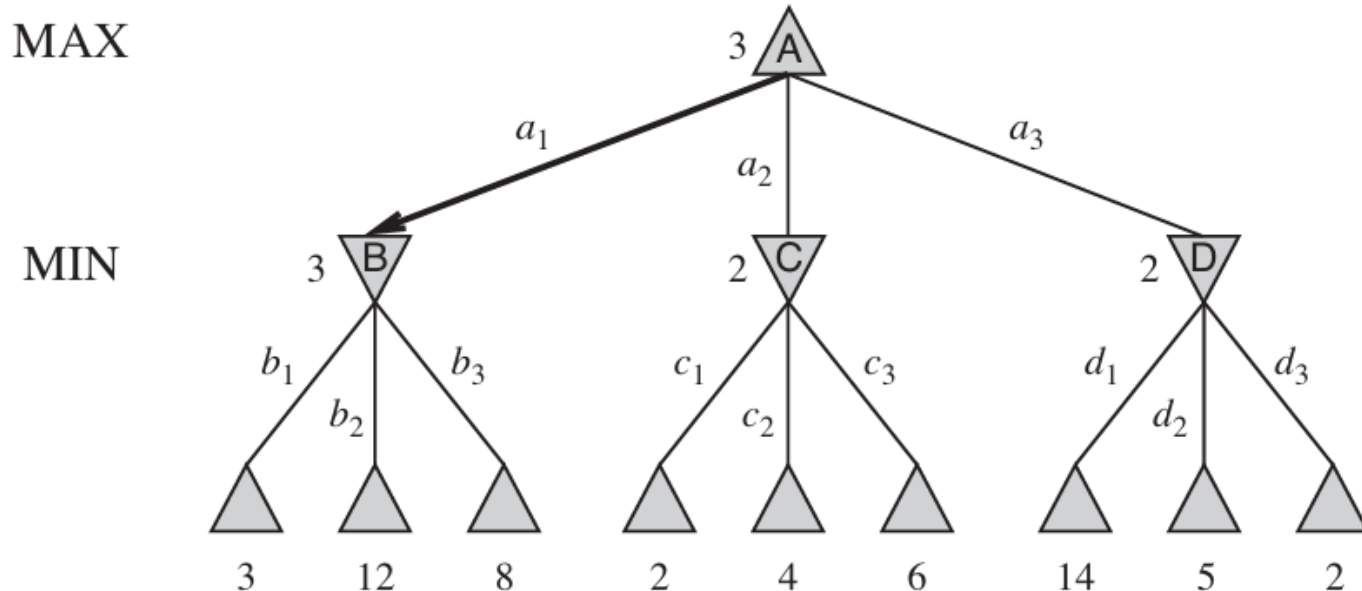


A simple 2-ply game



$\triangle$ nodes are “MAX nodes,” ∇ nodes are “MIN nodes.”

Minimax - A simple 2-ply game



MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Minimax Algorithm

function MINIMAX-DECISION(*state*) **returns** *an action*
 return $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(\text{state}, a))$

function MAX-VALUE(*state*) **returns** *a utility value*
 if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow -\infty$
 for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$
 return *v*

function MIN-VALUE(*state*) **returns** *a utility value*
 if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 $v \leftarrow \infty$
 for each *a* **in** ACTIONS(*state*) **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$
 return *v*

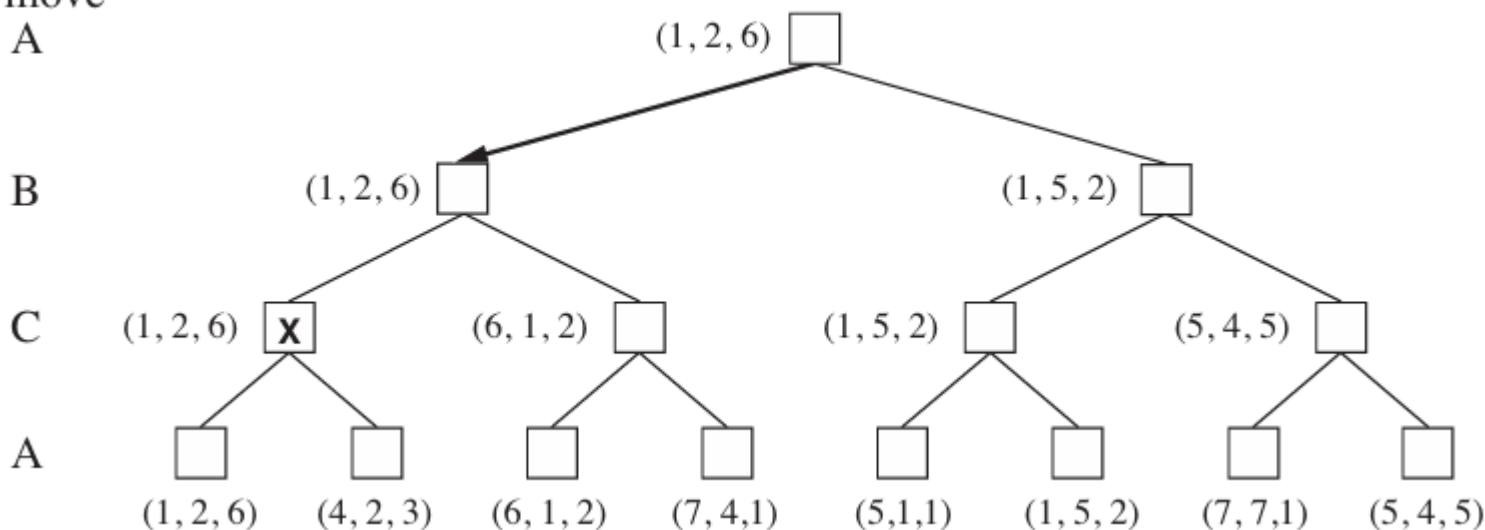
Minimax Algorithm

- Complete depth first exploration
- Depth m with b legal moves,
 - Time-complexity $O(b^m)$
 - Space-complexity $O(bm)$
- Chess: $m \approx 35$ on average: $50 \leq b \leq 100$
- Impractical for most games, but basis of other algorithm

Minimax for multiplayer

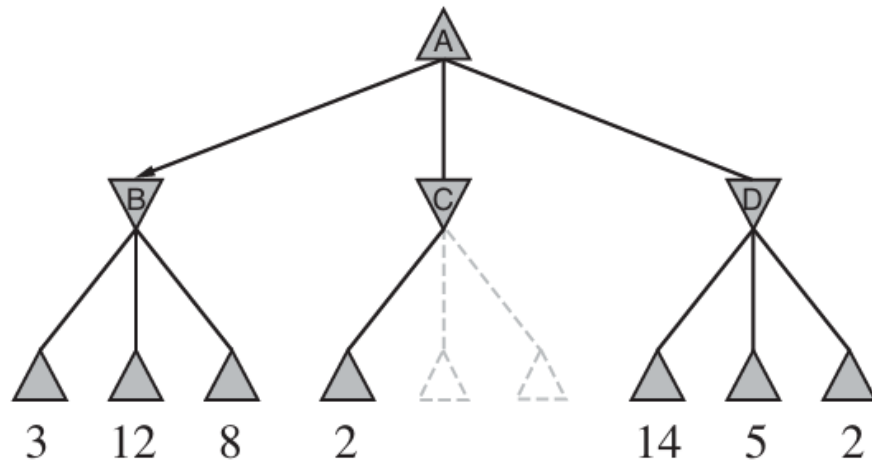
- Utility vectors instead of values

to move
A



Alpha-Beta pruning - Intuition

- Do we need to explore all nodes?

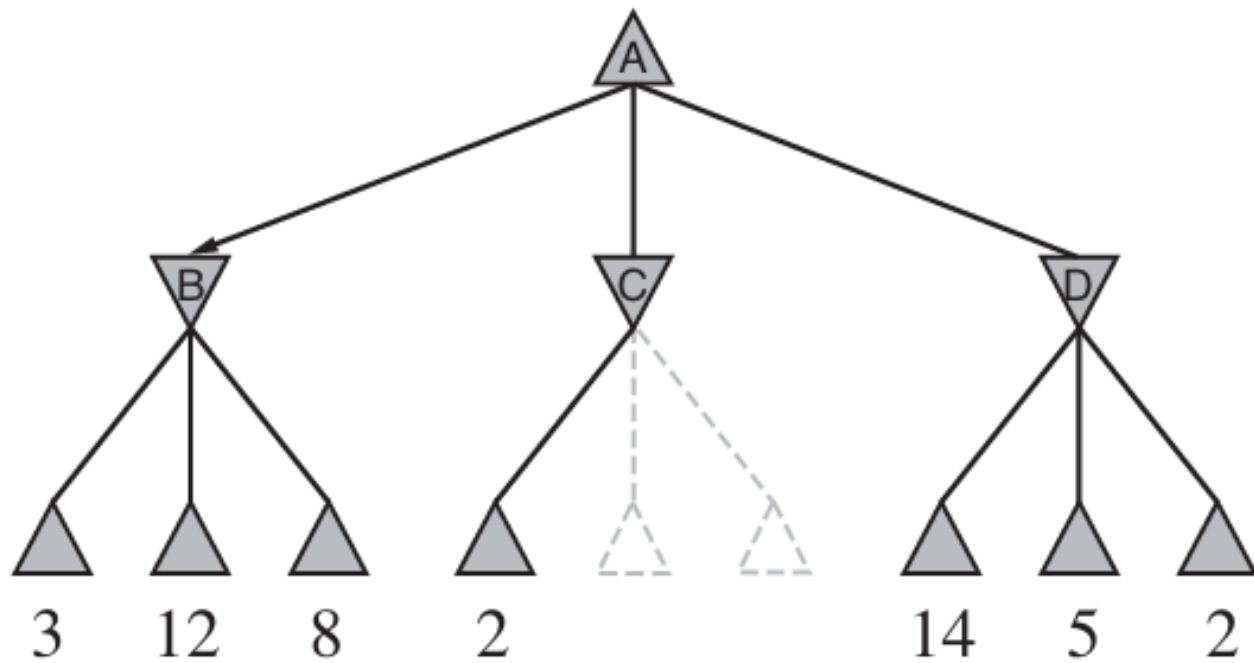


$$\begin{aligned}\text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) \\ &= \max(3, z, 2) \quad \text{where } z = \min(2, x, y) \leq 2 \\ &= 3.\end{aligned}$$

Alpha-Beta pruning

- Two values:
- α Value of best choice for MAX
- β Value of best choice for MIN
- Each node keeps track of its $[\alpha, \beta]$ values

Alpha-Beta pruning



Alpha-Beta pruning - Properties

- Pruning does not affect final outcome
- Sorting moves by result improves $\alpha - \beta$ performance
- Perfect ordering $O(b^{m/2})$
- Exercise on meta-reasoning

Alpha-Beta Algorithm

function ALPHA-BETA-SEARCH($state$) **returns** an action
 $v \leftarrow \text{MAX-VALUE}(state, -\infty, +\infty)$
 return the *action* in $\text{ACTIONS}(state)$ with value v

function MAX-VALUE($state, \alpha, \beta$) **returns** a utility value
 if $\text{TERMINAL-TEST}(state)$ **then return** $\text{UTILITY}(state)$
 $v \leftarrow -\infty$
 for each a **in** $\text{ACTIONS}(state)$ **do**
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
 if $v \geq \beta$ **then return** v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 return v

function MIN-VALUE($state, \alpha, \beta$) **returns** a utility value
 if $\text{TERMINAL-TEST}(state)$ **then return** $\text{UTILITY}(state)$
 $v \leftarrow +\infty$
 for each a **in** $\text{ACTIONS}(state)$ **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
 if $v \leq \alpha$ **then return** v
 $\beta \leftarrow \text{MIN}(\beta, v)$
 return v

Thank You

Dr. Rajiv Misra, Professor
Dept. of Computer Science & Engg.
Indian Institute of Technology, Patna
rajivm@iitp.ac.in



